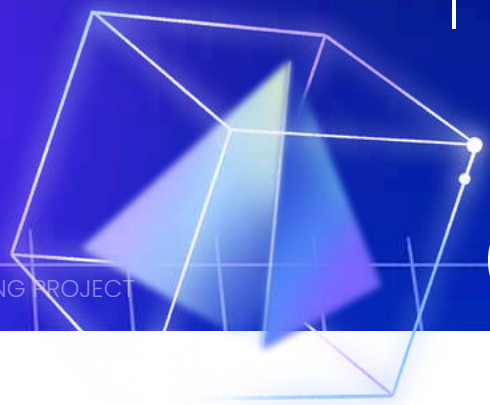
การพัฒนา MACHINE VISION ด้วย AI EDGE COMPUTING

B6418304 นายสุลพัส แสงเพชร
B6423568 นายกานต์พศุตม์ แสงทอง

ENG39 4102 : SEMINAR AND PRE-ELECTRONIC ENGINEERING PROJECT

TABLE OF CONTENTS

- ปัญหาและที่มาของโครงงาน
- วัตถุประสงค์ของโครงงาน
- ขอบเขตของโครงงาน
- Timeline
- What is Edge AI?
- Edge AI devices
- Why TensorFlow Lite matter?
- Block Diagram
- Google Coral
- OpenCV and Object Detection
- Basic information of Delta Arm
- งานที่ได้ทำไปแล้ว
- งานที่ยังทำไม่เสร็จ
- งานที่จะทำต่อในอนาคต



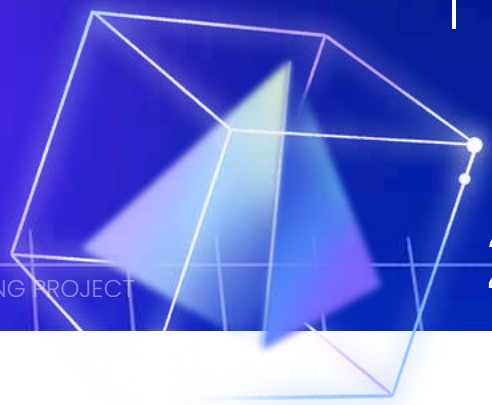
ปัญหาและที่มาของโครงงาน

ปัญหา ที่มา และเหตุผลของการพัฒนาโปรเจคนี้ เกิดจากประสิทธิภาพการประมวลผลของระบบ Machine Vision ในปัจจุบันนี้ ที่อาจเจอกับปัญหาด้านความล่าช้าในการประมวลผลภาพ หรือข้อมูลวัตถุที่มีปริมาณมาก โดยเฉพาะเมื่อข้อมูลถูกส่งไปยังศูนย์ข้อมูลหรือ Cloud เพื่อประมวลผล ทำให้เกิดความล่าช้าในการตอบสนอง อีกทั้งการประมวลผลภาพที่อยู่บน Cloud ยังอาจเผชิญกับข้อจำกัดด้าน Bandwidth และความเสถียรของการเชื่อมต่ออินเทอร์เน็ต รวมถึงความกังวลเกี่ยวกับความปลอดภัยของข้อมูล

ด้วยเหตุนี้ การพึ่งพาเทคโนโลยี Edge Computing จึงมีบทบาทสำคัญในการแก้ปัญหาเหล่านี้ เพราะสามารถประมวลผลภาพและข้อมูลได้ที่หน่วยอุปกรณ์ใกล้กับแหล่งข้อมูล(Local) โดยจะช่วยลดความล่าช้าและตอบสนองได้แบบเรียลไทม์ นอกจากนี้ การพัฒนาเทคโนโลยีนี้ยังมีเป้าหมายเพื่อเพิ่มประสิทธิภาพในการทำงานของระบบ Delta Arms ซึ่งเป็นหุ่นยนต์แขนกลที่ใช้งานในอุตสาหกรรมการผลิต เช่น การจัดเรียง การหยิบจับ โดยการนำ AI มาช่วยเสริมให้หุ่นยนต์สามารถวิเคราะห์ภาพวัตถุ ปรับตัวตามสภาพแวดล้อม ซึ่งจะช่วยเพิ่มความแม่นยำ และลดข้อผิดพลาดในการทำงานได้

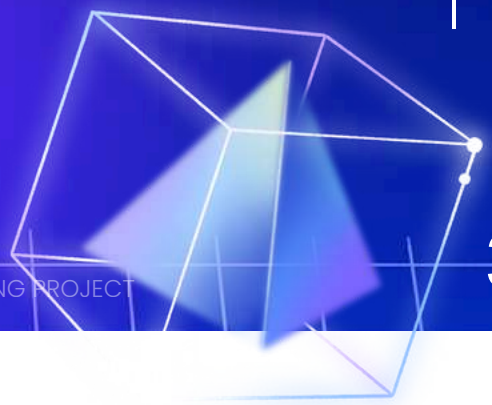
วัตถุประสงค์ของโครงงาน

- พัฒนา Machine Learning ที่ใช้ร่วมกับ Edge AI Accelerator ในการตรวจจับวัตถุ
- เพื่อประหยัดเวลาในการเก็บ หรือคัดแยก
- เพื่อลดต้นทุนด้านแรงงาน และค่าใช้จ่าย
- สามารถใช้งานได้ทุกที่



ขอบเขตโครงงาน

- รวบรวม Dataset และทำการ Train Model สำหรับใช้งาน
- ตรวจจับและระบุวัตถุได้แม่นยำ
- สามารถใช้งานร่วมกับ Edge AI Accelerator ได้
- สามารถสั่งให้แขนกล Delta Arm ไปหยิบจับวัตถุที่ตรวจจับได้



Timeline



		กรกฎาคม				สิงหาคม				กันยายน				พฤศจิกายน				ธันวาคม				มกราคม				กุมภาพันธ์				มีนาคม			
		1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
5	Timeline																																
5.1	ศึกษาเกี่ยวกับ Edge AI และวิธีการใช้งาน																																
5.2	ทดลองใช้ Trained Model กับตัว K210																																
5.3	ทดลองใช้ Trained Model กับตัว Google Coral																																
5.4	ศึกษา Delta Arm																																
5.5	ศึกษา Delta Arm Math																																
5.6	ศึกษา Raal Coordinate Math Calculation																																
5.7	ศึกษาการใช้ Webcam กับ Coral																																
5.8	ทดลองต่อจอ HDMI เข้า Coral																																
5.9	ทดลองใช้ Webcam																																
6	ทำ Custom Dataset																																
6.1	Train Custom Dataset																																
6.2	ทดลองใช้ Google Coral กับ Custom Dataset																																
6.3	ทดลองใช้ Google Coral รวมกับ Webcam เพื่อตรวจจับวัตถุ																																
6.4	ทดลองคำนวณ Real Coord. Math สำหรับ Delta Arms																																
6.5	ศึกษาการทำงานของ Delta Arm ของจริง																																
6.6	ทดลองใช้ Delta Arm เบื้องต้น																																
6.7	ทดลองปรับค่าต่างๆ เพื่อ Calibration ตัว Delta Arm กับตัว Code																																
6.8	ทดลองสั่ง Delta Arm ด้วย Code เบื้องต้น																																
6.9	ทดลองใช้ Delta Arm รวมกับ Custom Trained Model																																
7	เก็บรวบรวม Dataset ที่ใช้จริง																																
7.1	Train Dataset จริง ให้ได้ความแม่นยำสูงที่สุด																																
7.2	ใช้งาน Trained Dataset จริง รวมกับ Delta Arm																																
7.3	ปรับปรุงและแก้ไขส่วนที่บกพร่อง																																

WHAT IS EDGE AI?

Edge AI สามารถแยกออกเป็นสาขาของ **Edge Computing** ซึ่งเป็นแนวคิดการประมวลผลแบบใหม่ที่พัฒนาขึ้นเพื่อแก้ปัญหายุคของอุปกรณ์ Cloud ที่ใช้งาน IoT ซึ่งประสบปัญหาการรับข้อมูลจำนวนมากเกินไป (Data Overload) และค่าใช้จ่ายในการใช้บริการ Cloud ที่สูงเกินไป ด้วยเหตุนี้ Edge Computing จึงเข้ามาช่วยแก้ปัญหาโดยใช้การประมวลผลในแบบ Local เพื่อลดภาระของข้อมูลที่ส่งไปยังระบบ Cloud ส่วนกลาง ซึ่งเรียกว่า "Edge Network"

เมื่อปัญหาถูกแก้ไขแล้ว ก็สามารถนำสาขาของ **ปัญญาประดิษฐ์ (AI: Artificial Intelligence)** มาผสานเข้าด้วยกันได้ โดย AI เป็นเทคโนโลยีที่เราสามารถฝึกฝนด้วยข้อมูล ทำให้เครื่องมีความสามารถในการแก้ไขปัญหาเช่นเดียวกับมนุษย์ โดยเทคโนโลยีที่ได้รับความนิยมมากที่สุดคือ Machine Learning (ML) ซึ่งมีวัตถุประสงค์เพื่อให้เครื่องจักรเลียนแบบกระบวนการเรียนรู้ของมนุษย์โดยใช้ข้อมูลจำนวนมาก ซึ่งจะทำให้เครื่องสามารถมองเห็น ตรวจจับ และพยายามทำนายว่า วัตถุนั้นคืออะไร

สรุปแล้ว Edge AI คือการนำแอปพลิเคชัน AI มาใช้งานในอุปกรณ์ต่างๆ และการประมวลผลนั้นจะเกิดขึ้นแบบ Local และสามารถติดตั้งได้ทุกที่เหมือนกับคอมพิวเตอร์ขนาดเล็ก — (**https://dl.acm.org/doi/10.1145/3555802**)

EDGE AI DEVICES

เนื่องจากการใส่ตัว AI เข้าไป และการนำไปใช้งานในอุปกรณ์ต่าง ๆ ก่อให้เกิดปัญหาเกี่ยวกับ CPU ที่ไม่สามารถประมวลผลและรับข้อมูลได้เร็วพอในแบบ real-time ผู้ผลิตจึงได้พัฒนาอุปกรณ์ที่มี CPU ที่เร็วขึ้นกว่าเดิม เรียกว่า TPU (Tensor Processing Unit) หรือ NPU (Neural Processing Unit) และทำให้รองรับการใช้งานกับ AI โดยเฉพาะเข้าไปในอุปกรณ์เหล่านี้ด้วย โดยอุปกรณ์เหล่านี้จำเป็นต้องใช้ไฟล์ Model ที่ Train ขึ้นมาแบบขนาดเล็ก โดยการแปลงตัว Model ขนาดดั้งเดิม ให้มีขนาดเล็กลง แต่ยังคงความสามารถในการทำนาย ในรูปแบบของ **TensorFlow Lite** และนี่คือตัวอย่างบางส่วนของอุปกรณ์ที่ใช้งาน



Google Coral AI Development Board Mini
by Google



Jetson Nano Developer Kit
by NVIDIA



Hailo NPU for Raspberry Pi 5
by Hailo

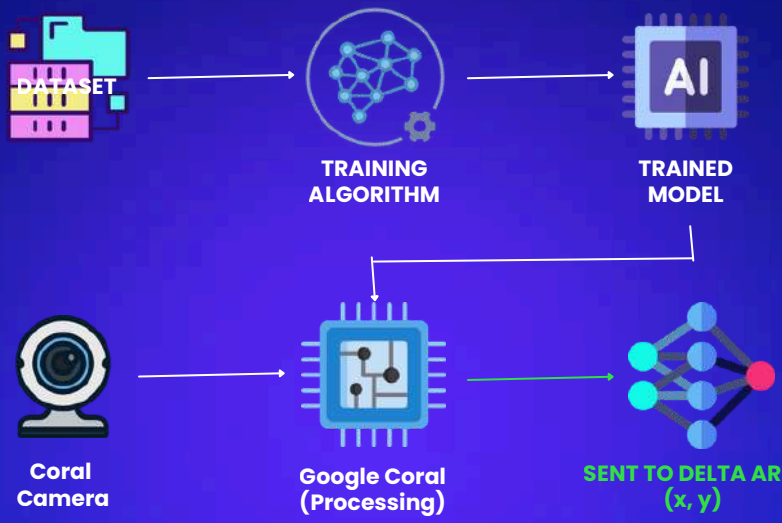
Why is TensorFlow Lite matter?

TensorFlow Lite มีความสำคัญอย่างยิ่งต่อการประมวลผลของ Edge Computing เนื่องจากไฟล์ชนิดนี้ได้รับการปรับแต่งเป็นพิเศษสำหรับการรันโมเดล Machine Learning บนอุปกรณ์ที่มีทรัพยากรจำกัด เช่น สมาร์ทโฟน Embedded Systems และอุปกรณ์ IoT โดยจะแตกต่างจาก TensorFlow แบบปกติ เนื่องจาก TensorFlow Lite ทำให้โมเดลมีขนาดเล็กลง (Model Quantization) เพื่อช่วยลดขนาดของโมเดลและปรับปรุงประสิทธิภาพ ทำให้สามารถประมวลผลแบบเรียลไทม์บนอุปกรณ์ที่มีความสามารถในการประมวลผลต่ำได้

ในขณะที่เมื่อเทียบกับ YOLO (You Only Look Once) ที่ถูกออกแบบมาสำหรับการตรวจจับวัตถุแบบเรียลไทม์ แต่แลกมาด้วยขนาดโมเดลที่ใหญ่ ต้องใช้ความแรงของเครื่องในการประมวลผล และอาจเกินความจำเป็นสำหรับตัว Edge Device เมื่อเทียบ TensorFlow Lite ที่มีความยืดหยุ่นมากกว่า รองรับโมเดลและแอปพลิเคชันที่หลากหลาย ไม่จำกัดแค่การตรวจจับวัตถุเพียงอย่างเดียว ทำให้ TensorFlow Lite เป็นทางเลือกที่เหมาะสมที่สุดในการใช้งาน AI ที่มีประสิทธิภาพและหลากหลายบน Edge Device มากกว่า



BLOCK DIAGRAM



Google Coral



Coral Dev Board Mini เป็นคอมพิวเตอร์บอร์ดเดี่ยวที่สามารถใช้งาน Machine Learning ได้อย่างรวดเร็ว โดยอุปกรณ์นี้ถูกออกแบบมาพร้อมกับตัว Accelerator Module (โมดูลที่ติดตั้งเพื่อใช้งาน Edge TPU) แต่ก็ยังที่ทำงานได้ดีเมื่อเทียบกับ USB Accelerator ที่เป็นตัวต่อแยกต่างหากที่ดีกว่า แต่แค่ Module ที่ติดมาก็สามารถนำไปใช้กับโปรเจค Machine Learning ที่ทำงานบนอุปกรณ์ต่างๆ ได้ ตัวEdge TPU ที่ติดตั้งบนบอร์ดนี้สามารถทำการประมวลผลได้ 4 ล้านล้านคำสั่งต่อวินาที (TOPS) โดยใช้พลังงาน 0.5 วัตต์ต่อ TOPS (หรือ 2 TOPS ต่อวัตต์) นอกจากนี้ยังรองรับ TensorFlow Lite พร้อมใช้งานร่วมกับภาษา Python ปกติ

OpenCV and Object Detection



OpenCV (Open Source Computer Vision Library) คือ ไลบรารี Open-Source ที่ใช้สำหรับการพัฒนาแอปพลิเคชันทางด้าน Computer Vision และการประมวลผลภาพ (Image Processing) โดยถูกพัฒนาและเปิดตัวครั้งแรกในปี 2000 โดยบริษัท Intel โดย OpenCV มีเป้าหมายเพื่อช่วยให้นักพัฒนาสามารถสร้างแอปพลิเคชันที่เกี่ยวข้องกับการวิเคราะห์ภาพและวิดีโอ เช่น การตรวจจับวัตถุ (Object Detection), การจดจำใบหน้า (Face Recognition), การติดตามการเคลื่อนไหว (Motion Tracking) และอื่นๆ

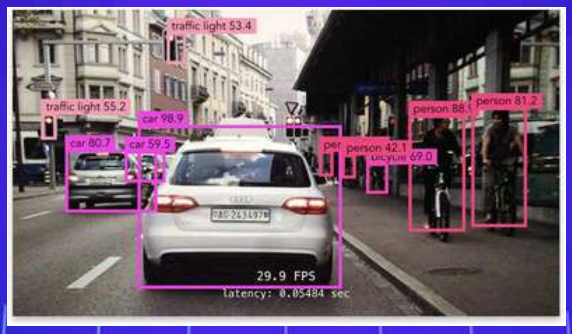
OpenCV รองรับการทำงานกับหลายภาษา เช่น Python, C++, Java และสามารถใช้งานได้ทั้งบนระบบปฏิบัติการ Windows, macOS, Linux รวมถึงแพลตฟอร์มบนอุปกรณ์เคลื่อนที่ เช่น Android และ iOS

ด้วยความสามารถในการจัดการภาพและวิดีโออย่างมีประสิทธิภาพและรวดเร็ว OpenCV จึงถูกใช้อย่างแพร่หลายในงานวิจัย การพัฒนาโปรเจคด้าน AI, การสร้างหุ่นยนต์อัตโนมัติ และการประยุกต์ใช้ในอุตสาหกรรมต่าง ๆ ที่ต้องการการประมวลผลภาพในแบบเรียลไทม์

OpenCV and Object Detection

การตรวจจับวัตถุ (Object Detection) คือ เทคโนโลยีที่ใช้ในการระบุวัตถุในภาพหรือวิดีโอ โดยจะบอกว่ามีวัตถุอะไรบ้าง และบอกตำแหน่งที่แน่นอนของวัตถุ โดยการตรวจจับวัตถุจะใช้โมเดล เช่น SSD (Single Shot Multibox Detector) หรือ YOLO (You Only Look Once) ที่สามารถประมวลผลภาพ และระบุตำแหน่งของวัตถุได้ในขั้นตอนเดียว

ในขั้นตอนการทำงานของ Object Detection ระบบจะทำการแบ่งภาพออกเป็นกรอบย่อย ๆ และตรวจสอบวัตถุในแต่ละกรอบ จากนั้นโมเดลจะทำการคำนวณคะแนนความมั่นใจ (confidence score) เพื่อบอกว่าวัตถุที่พบในกรอบนั้นคืออะไร ตัวอย่างเช่น ในภาพที่มีคนและรถยนต์ โมเดลจะบอกว่าในกรอบนี้คือ "คน" ด้วยความมั่นใจ 95% และในกรอบนี้คือ "รถยนต์" ด้วยความมั่นใจ 90% สิ่งนี้ช่วยให้ระบบสามารถนำข้อมูลที่ได้ไปใช้ประโยชน์ในงานต่าง ๆ ได้



Basic information of Delta Arm

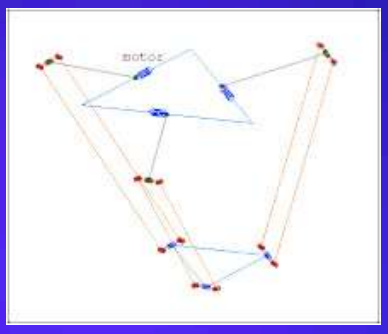
หุ่นยนต์แขนกลเดลต้า หรือ หุ่นยนต์ขนาน (Parallel Robot) หรือหุ่นยนต์แมงมุม เป็นหุ่นยนต์อุตสาหกรรมที่มีโครงสร้างทางเรขาคณิตรูปแบบขนาน ซึ่งประกอบด้วยแขนกลสามแกนเชื่อมต่อกันเป็นรูปสามเหลี่ยม ทำให้มีลักษณะคล้ายขาแมงมุม จึงมักเรียกกันติดปากว่า "หุ่นยนต์แมงมุม" หุ่นยนต์ชนิดนี้ได้รับการออกแบบมาเพื่อให้มีประสิทธิภาพสูงในด้านความเร็ว ความแม่นยำ และความคล่องตัว เหมาะสำหรับงานที่ต้องการการหยิบจับและวางวัตถุขนาดเล็กซ้ำๆ อย่างรวดเร็ว หุ่นยนต์เดลต้าทำงานโดยอาศัยการควบคุมมอเตอร์ที่ติดตั้งอยู่ที่ฐานของแต่ละแกน เมื่อได้รับคำสั่งจากระบบควบคุม มอเตอร์จะขับเคลื่อนให้แกนหมุน ทำให้ปลายแขนเคลื่อนที่ไปยังตำแหน่งที่ต้องการได้อย่างแม่นยำและรวดเร็ว โดยระบบควบคุมจะคำนวณตำแหน่งและทิศทางของปลายแขนจากข้อมูลที่ได้จากเซ็นเซอร์ต่างๆ

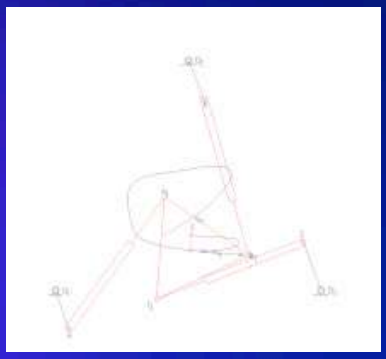


Basic information of Delta Arm

แนวคิดหลักของหุ่นยนต์เดลต้าคือการใช้รูปสี่เหลี่ยมด้านขนาน ซึ่งมีขอบเขตการเคลื่อนไหวของตัวมัน กล่าวคือ เคลื่อนที่ได้เฉพาะในแนวแกน X, Y หรือ Z โดยไม่มีการหมุน เนื่องจากตัวขับเคลื่อนทั้งหมดอยู่ที่ฐาน แขนจึงสามารถทำจากวัสดุชนิดน้ำหนักเบา ส่งผลให้ชิ้นส่วนที่เคลื่อนไหวของหุ่นยนต์เดลต้ามีโมเมนต์ความเฉื่อยต่ำ ทำให้สามารถเคลื่อนที่ได้เร็วมากและมีความเร่งสูง



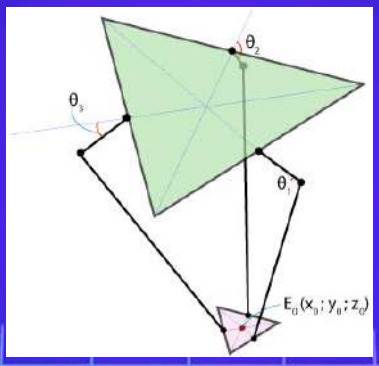




Basic information of Delta Arm

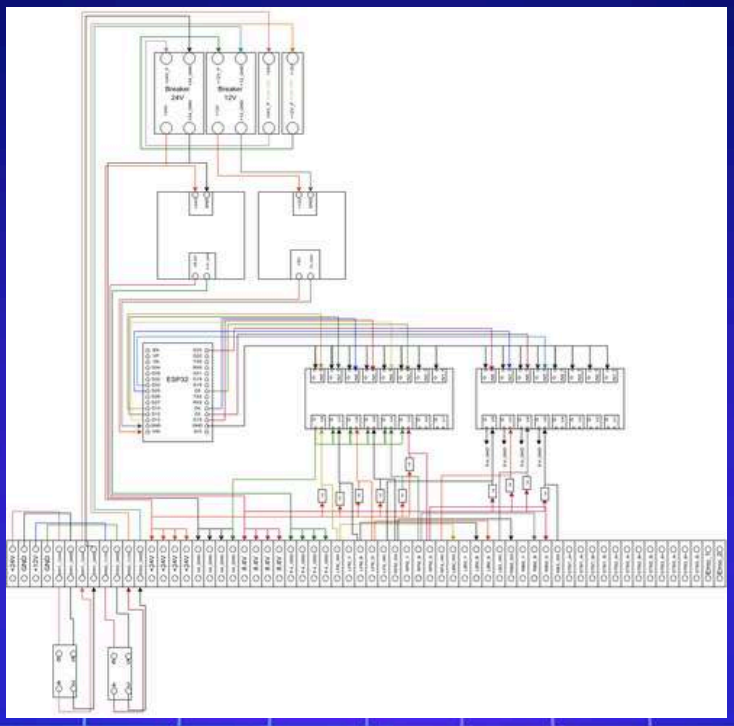
หากเราต้องการควบคุม โดยหากเรารู้ตำแหน่งที่ต้องการของ End Effector(E_0) ตัวอย่างเช่น หากต้องการจับวัตถุในจุดที่มีพิกัด X, Y, Z ก็จำเป็นต้องกำหนดมุมที่สอดคล้องกันของแขนทั้งสามแขน(มุมข้อต่อ) เพื่อตั้งมอเตอร์ และ End Effector(E_0) ในตำแหน่งที่เหมาะสมสำหรับการหยิบ โดยกระบวนการในการกำหนดดังกล่าวเรียกว่า **"Inverse Kinematics"**
และหากเรารู้มุมข้อต่อ ตัวอย่างเช่น หากทราบค่าจาก ตัว Encoders ก็สามารถกำหนดตำแหน่งของ End Effector(E_0) โดยกระบวนการในการกำหนดดังกล่าวเรียกว่า **"Forward Kinematics"**

เพื่อให้เห็นภาพ อันที่เป็นสีเขียวคือตัว Motor และอันที่เคลื่อนที่พร้อม End Effector คือ สีชมพู มุมข้อต่อคือ theta1, theta2 และ theta3 และจุด E_0 คือตำแหน่ง End Effector ที่มีพิกัด (x0, y0, z0) เพื่อแก้ปัญหา **"Inverse Kinematics"** เราต้องสร้างฟังก์ชันที่มีพิกัด E_0 (x0, y0, z0) เป็นพารามิเตอร์ซึ่งส่งคืน (theta1, theta2, theta) หากเป็น **"Forward Kinematics"** จะได้รับ (theta1, theta2, theta3) และส่งกลับ (x0, y0, z0)



Basic information of Delta Arm

Diagram



Basic information of Delta Arm



57BYG250C-8 Stepper Motor

- **ประเภท:** Hybrid Stepper Motor
- **ขนาดเฟรม:** 57 มม. (ขนาดเฟรม 57 หมายถึงขนาดของตัวมอเตอร์ 57x57 มม.)
- **จำนวนเฟส:** 2 เฟส
- **จำนวนสาย (Wires):** 8 สาย (สำหรับควบคุมทั้งสองเฟส) ทำให้สามารถเชื่อมต่อแบบ Unipolar หรือ Bipolar
- **มุมการหมุนต่อสเต็ป (Step Angle):** 1.8 องศาต่อสเต็ป มอเตอร์แบบนี้จะมี 200 สเต็ปต่อการหมุน 1 รอบ (360 องศา)
- **แรงบิดค้าง (Holding Torque):** 2.5 นิวตันเมตร (Nm)
- **แรงดันไฟฟ้า (Voltage):** 3.6 V
- **กระแสไฟฟ้า (Current):** 3.0 แอมแปร์ต่อเฟส (A/Phase)กระแสไฟฟ้าที่มอเตอร์ต้องการสำหรับแต่ละเฟส

Basic information of Delta Arm

HC-SR04 (Ultrasonic Distance Sensor)



เป็นเซ็นเซอร์ระยะทาง (Ultrasonic Distance Sensor) ที่ใช้คลื่นเสียงความถี่สูงในการวัดระยะทาง โดยทั่วไปจะใช้ในโครงการหุ่นยนต์และระบบตรวจจับระยะทางต่างๆ

- **แหล่งจ่ายไฟ:** 5V
- **ระยะการวัด:** 2 cm ถึง 400 cm
- **ความละเอียด:** ประมาณ 3 mm
- **มุมการวัด:** ประมาณ 15 องศา
- **ความถี่:** 40 kHz

$$\text{ระยะทาง} = \frac{\text{เวลา} \times \text{ความเร็วเสียง}}{2}$$

HC-SR04

sketch_sep18a | Arduino IDE 2.2.1

File Edit Sketch Tools Help

DOIT ESP32 DEVKIT V1

sketch_sep18a.ino

```
#include <Wire.h>
#define echoPin 2              // CHANGE PIN NUMBER HERE IF YOU WANT TO USE A DIFFERENT PIN
#define trigPin 4              // CHANGE PIN NUMBER HERE IF YOU WANT TO USE A DIFFERENT PIN
long duration, distance;
void setup(){
  Serial.begin (9600);
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
}
void loop(){
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  duration = pulseIn(echoPin, HIGH);
  distance = duration / 58.2;
  String disp = String(distance);

  Serial.print("Distance: ");
  Serial.print(disp);
  Serial.println(" cm");
  delay(1000);
}
```

COM5

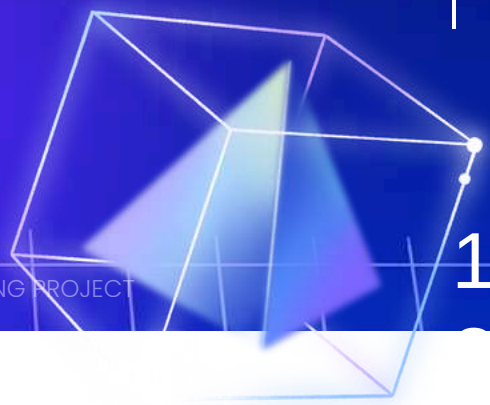
Send

```
Distance: 183 cm
Distance: 183 cm
Distance: 183 cm
Distance: 183 cm
Distance: 183 cm
Distance: 183 cm
Distance: 183 cm
```

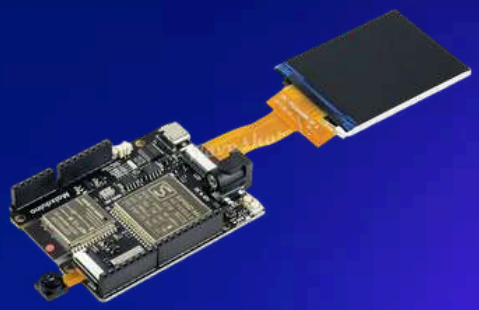
Autoscroll No line ending 9600 baud

งานที่ได้ทำไปแล้ว

- นำ Trained-Model ลง Edge K210
- นำ Trained-Model ลง Edge Coral
- เมื่อตรวจจับบอกตำแหน่ง x, y
- K210 ด้วย Custom Model
- Coral ด้วย Custom Model
- Quantization Model
- Evaluate Model
- ใช้ Coral กับ กล้อง
- ปรับแก้ FPS Coral Camera
- Math Real Rotation(Theta). from x, y bbox
- ตัดชิ้นงานฟันเฟือง และประกอบ



นำ Trained-Model ลง Edge K210



Sipeed Maixduino K210

Maixduino มาพร้อมกับ Sipeed M1 ที่รวมโมดูล Kendryte K210 และโมดูล ESP32 สำหรับการเชื่อมต่อ WiFi และ Bluetooth ในรูปแบบของ Arduino UNO อุปกรณ์นี้ออกแบบมาเพื่อให้สามารถรัน AI ที่เอดจ์ (Edge) โดยมอบประสิทธิภาพสูงในขนาดที่กะทัดรัดต่างจาก Google Edge TPU ที่ทำหน้าที่เป็นตัวเร่งประสิทธิภาพ (accelerator) Sipeed MAIX ทำหน้าที่เป็นตัวควบคุมหลัก (master controller) ซึ่งช่วยให้สามารถสร้างผลลัพธ์ที่ใช้พลังงานน้อยลง และมีต้นทุนต่ำลงได้ นอกจากนี้ชุดนี้ยังมาพร้อมกับหน้าจอ TFT ขนาด 2.4 นิ้ว ทำให้สามารถแสดงผลได้



นำ Trained-Model ลง Edge K210



M5StickV

M5StickV เป็นเวอร์ชันขนาดเล็กของ Sipeed Maixduino K210 โดยได้ตัดโมดูล ESP32 ออก และมีเพียงกล้องขนาดเล็ก พร้อมปุ่ม GPIO 2 ปุ่มและหน้าจอ LCD ดังนั้น กระบวนการใช้งานจึงเหมือนกับ Sipeed Maixduino K210

นำ Trained-Model ลง Edge K210







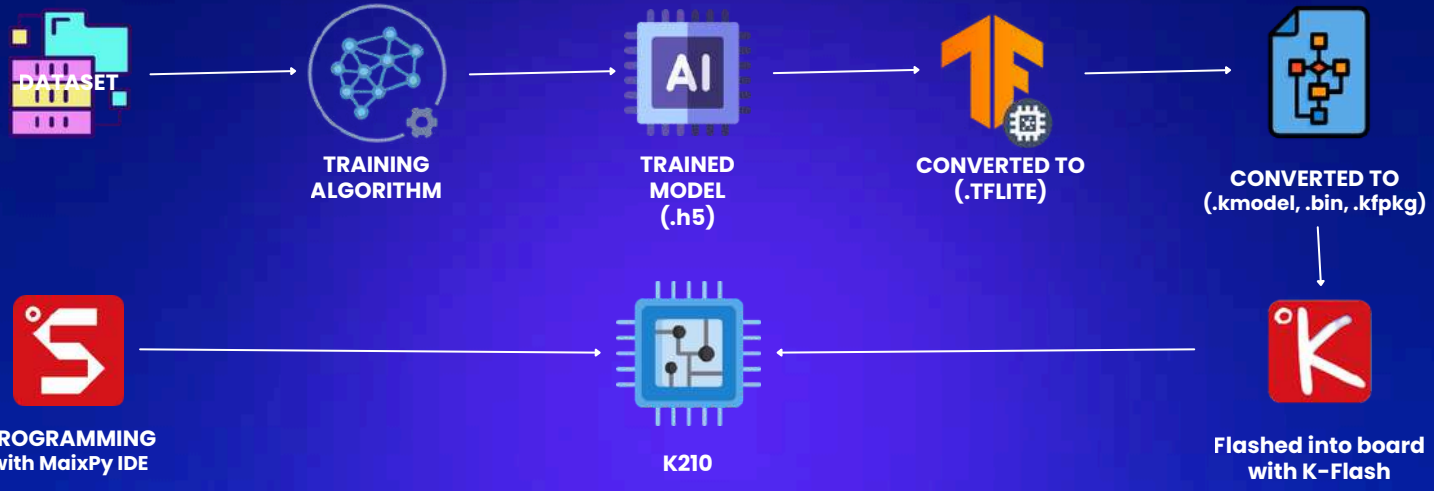
Programming Languages

ตระกูล K210 แทบทั้งหมด มักจะใช้ภาษา “Micropython” เนื่องจากโมดูลนี้มีขนาดเล็กกว่าคอมพิวเตอร์ปกติ จึงไม่สามารถใช้งานกับ Python เวอร์ชั่นปกติที่ต้องการการใช้งาน RAM มากได้ นอกจากนี้ Sipeed Maixduino K210 และ M5StickV ยังมีคอมไพเลอร์และโปรแกรม IDE ของตนเองชื่อว่า “MaixPy” ที่ทำให้การเขียนโค้ดและการทดสอบบนโมดูลนี้ง่ายขึ้น

เมื่อเสร็จสิ้นกระบวนการเขียนโค้ด จะต้องทำการแฟลชลงบอร์ดคล้ายกับ ESP32 แต่โมดูลนี้มีโปรแกรมมีคนสร้างขึ้นเพื่อช่วยในการแฟลชสำหรับชิป Kendryte K210 ทั้งหมด ชื่อว่า “K-flash” เพื่อจัดการกระบวนการแฟลชเข้าสู่บอร์ดได้อย่างสะดวก

นำ Trained-Model ลง Edge K210 & เมื่อตรวจจับบอกตำแหน่ง x, y

Process Diagram



นำ Trained-Model ลง Edge K210 & เมื่อตรวจจับบอก ตำแหน่ง x, y

Sipeed Maixduino K210 Result





n Trained-Model ลง Edge K210 & เมื่อตรวจจับบอกตำแหน่ง x, y

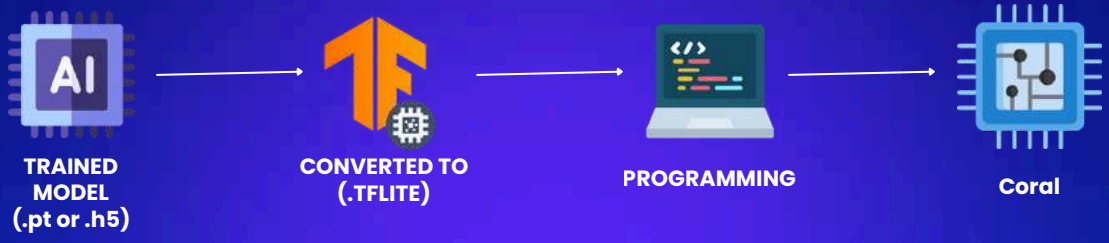
M5StickV Result





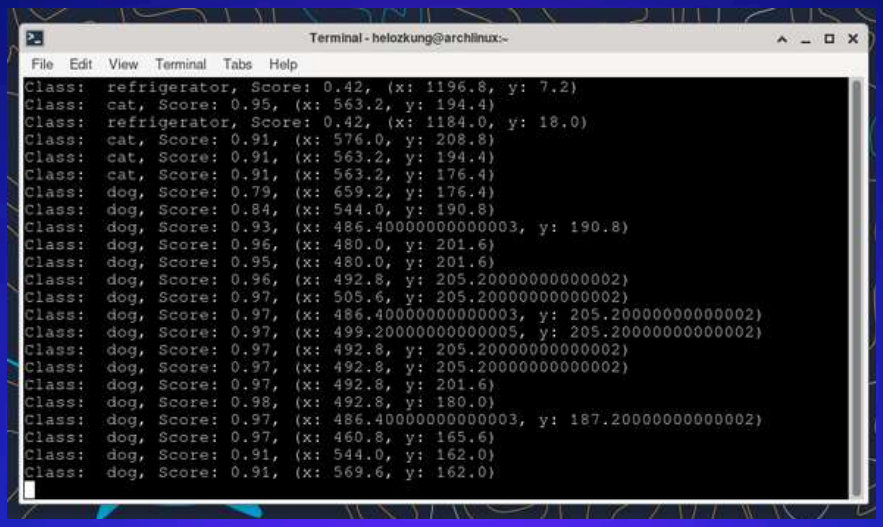
n Trained-Model ลง Edge Coral & เมื่อตรวจจับบอกตำแหน่ง x, y

Process Diagram



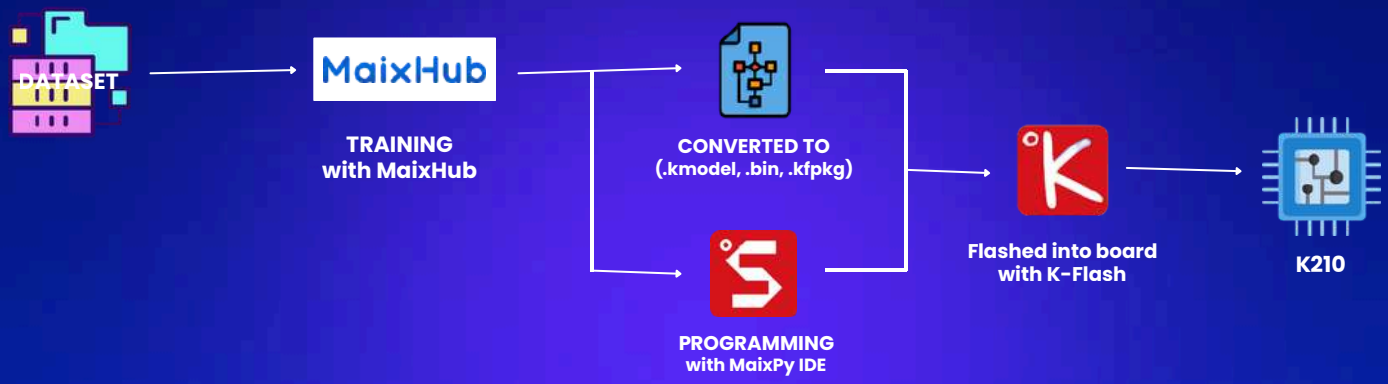
n Trained-Model ลง Edge Coral & เมื่อตรวจจับบอกตำแหน่ง x, y

Coral Result



K210 ด้วย Custom Model

Diagram



K210 ด้วย Custom Model

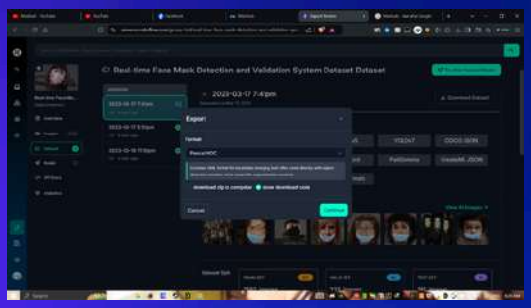
Dataset



Dataset ที่นำมาใช้ คือ **“Real-time Face Mask Detection and Validation System Dataset Computer Vision Project”** จาก **Roboflow** เพื่อนำมาใช้ Train model โดย Download มาในรูปแบบของ **“Pascal VOC format”**. Dataset นี้มีทั้งหมด **3 classe** คือ **proper_mask**, **improper_mask** และ **no_mask** ทั้งหมด **8,280** ไฟล์:

- **images**: **4,140** ไฟล์
- **labeling(XML)**: **4,140** ไฟล์



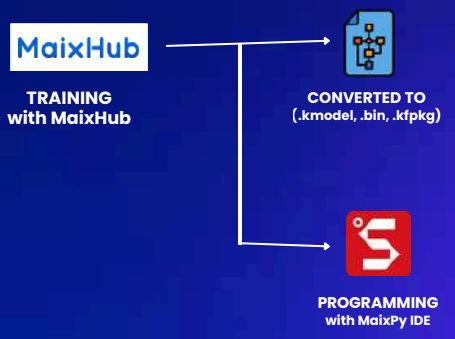


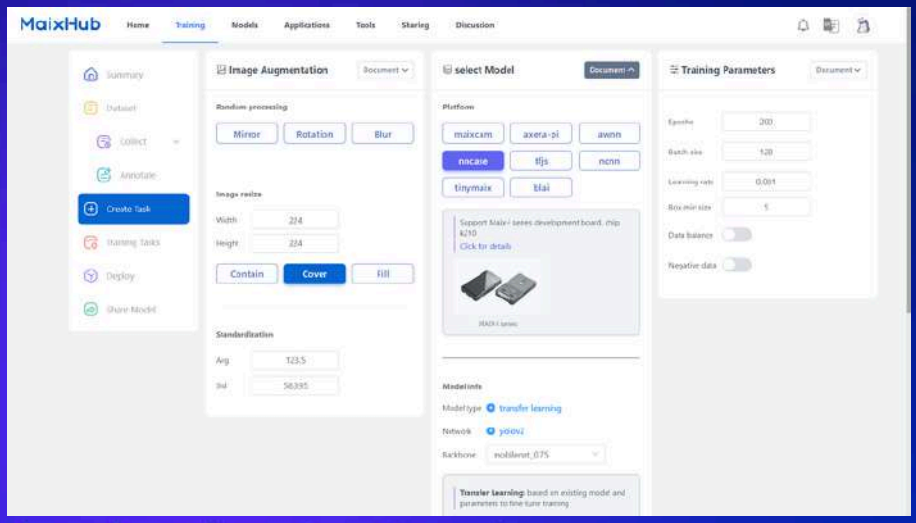
https://universe.roboflow.com/group-tbd/real-time-face-mask-detection-and-validation-system-dataset/dataset/4

K210 ด้วย Custom Model

Process Training

MaixHub เป็นแพลตฟอร์ม Zero Code Training ฟรีที่พัฒนาโดยเจ้าของ MaixPy ซึ่งออกแบบมาให้รองรับและทำงานร่วมกับชิป K210 ได้อย่างมีประสิทธิภาพมากขึ้น ข้อดีสำคัญของ MaixHub คือมันมาพร้อมกับตัวแปลงสำหรับ K210 ในตัว ดังนั้นก็ไม่จำเป็นต้องทำการแปลงข้อมูลด้วยตัวเอง กระบวนการจะทำงานและแปลงไฟล์อัตโนมัติให้เข้ากับชิป K210 ได้โดยตรง



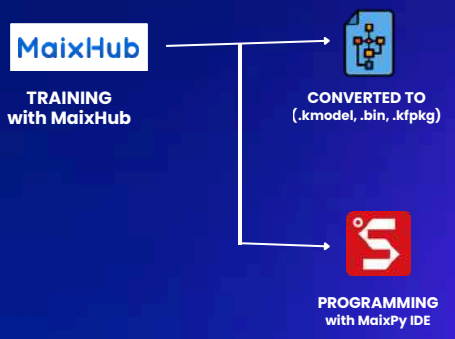


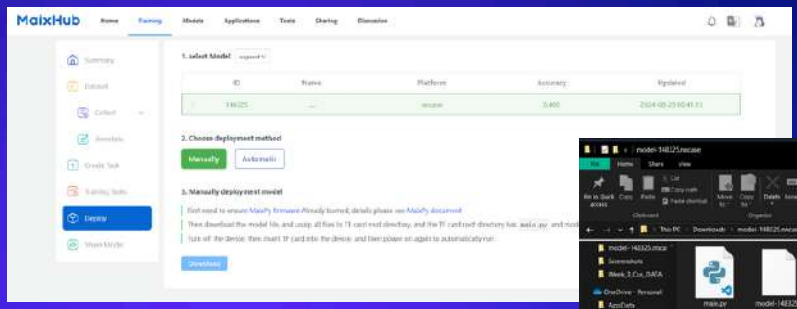
K210 ด้วย Custom Model

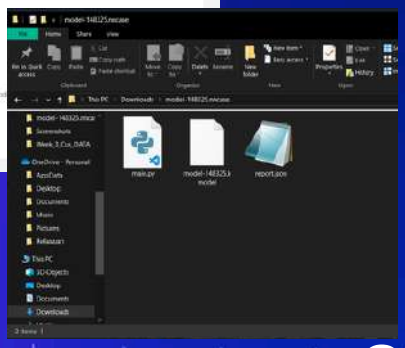
Process Training

หลังจากการฝึกโมเดลเสร็จสิ้น จะสามารถดาวน์โหลดไฟล์ ZIP ได้โดยตรง ซึ่งภายในจะมีไฟล์สำคัญ 2 ไฟล์ดังนี้:

- **main.py:** เป็นไฟล์โค้ดที่ใช้สำหรับกำหนดค่า anchors จากโมเดลที่ถูกฝึกมาแล้ว รวมถึงโค้ดที่จำเป็นสำหรับการทำงานบนบอร์ด K210
- **model.kmodel**: เป็นไฟล์โมเดลที่ถูกแปลงแล้ว (Kendryte Model) ซึ่งใช้สำหรับแฟลชลงบอร์ด K210







K210 ด้วย Custom Model

Result

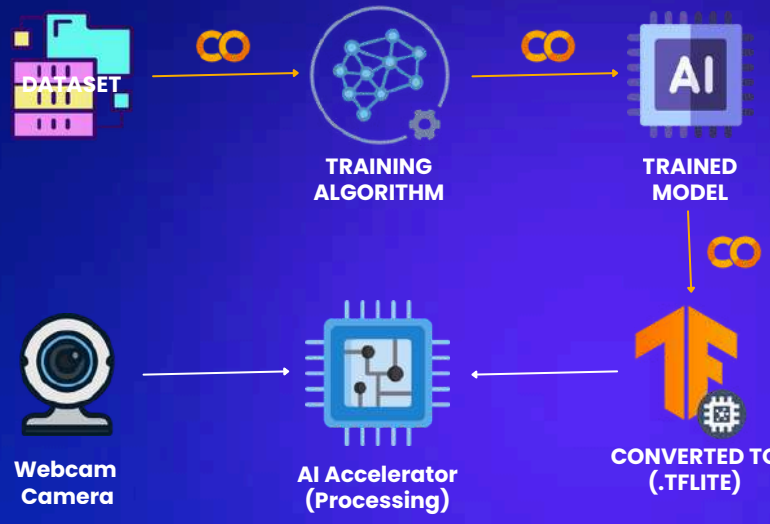






Coral ด้วย Custom Model

Process Diagram



USER's Computer*

*เนื่องจากตัว Edge AI Accelerator ไม่มี GPU ที่แรงพอสำหรับ Train ตัว Model แต่มี TPU, NPU ที่เป็นตัว Processor ที่มีความสามารถในการประมวลผล และ ทำนายได้รวดเร็วกว่า

Edge Devices Computer with AI Accelerator

Coral ด้วย Custom Model

Process Dataset

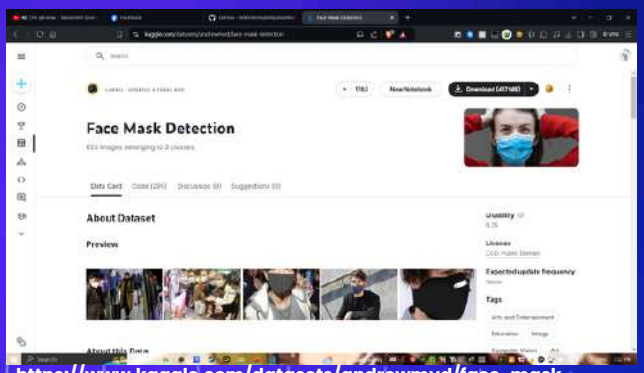


ขอขอบคุณอย่างยิ่งสำหรับ Dataset ชื่อ Face Mask Detection จาก Kaggle ที่ให้ Download มาสำหรับการฝึกโมเดล โดยชุดข้อมูลนี้ประกอบด้วย 3 คลาส ได้แก่ **with_mask**, **without_mask** และ **mask_wared_incorrect** เป็นจำนวนทั้งหมด **1,706** ไฟล์:

- **images**: **853** ไฟล์
- **labeling(XML)**: **853** ไฟล์

และได้แบ่ง Dataset ออกเป็นส่วนสำหรับการฝึก(Training) และการตรวจสอบ (validation) ดังนี้:

- **training_image, training_labeling(XML)**: อย่างละ **684** ไฟล์
- **validation_image, validation_labeling(XML)**: อย่างละ **169** ไฟล์



https://www.kaggle.com/datasets/andrewmvd/face-mask-detection

Coral ด้วย Custom Model

Process Training

ผลลัพธ์การ Train



Quantization Model

การ **Quantization** คือ กระบวนการที่ช่วยลดขนาดของโมเดลและเพิ่มประสิทธิภาพในการทำงาน โดยเฉพาะบนอุปกรณ์ที่มีข้อจำกัดด้านทรัพยากร เช่น สมาร์ทโฟน, ไมโครคอนโทรลเลอร์ หรือ Edge Devices อย่าง Edge TPU กระบวนการนี้ช่วยลดการใช้หน่วยความจำ, พลังงาน และเวลาในการประมวลผล โดยทำให้โมเดลสามารถทำงานได้รวดเร็วขึ้น แต่ยังคงรักษาความแม่นยำให้อยู่ในระดับที่ยอมรับได้

หลักการของ Quantization โดยปกติแล้วโมเดล Machine Learning จะใช้ตัวเลขแบบ floating-point 32 บิต (FP32) ซึ่งแม่นยำมากแต่ใช้ทรัพยากรเยอะ การทำ Quantization จะลดขนาดของตัวเลขที่ใช้ในการคำนวณจาก 32 บิตเหลือเป็นขนาดที่เล็กลง เช่น 16 บิต (FP16) หรือ 8 บิต (INT8) ซึ่งทำให้โมเดลเบาขึ้นและประหยัดพลังงานมากขึ้น การลดจำนวนบิตจะลดความแม่นยำในการแสดงผลทางคณิตศาสตร์ลงเล็กน้อย แต่ความแม่นยำของโมเดลโดยรวมยังคงใกล้เคียงกับโมเดลที่ใช้ FP32

```python
#get input details for model
interpreter = Interpreter(model_path=PATH_TO_MODEL)
interpreter.allocate_tensors()
input_details = interpreter.get_input_details()
output_details = interpreter.get_output_details()
height = input_details[0]['shape'][1]
width = input_details[0]['shape'][2]

import random

def representative_data_gen():
  dataset_list = quant_image_list
  quant_num = 300
  for i in range(quant_num):
    pick_me = random.choice(dataset_list)
    image = tf.io.read_file(pick_me)

    if pick_me.endswith('.jpg') or pick_me.endswith('.JPG'):
      image = tf.io.decode_jpeg(image, channels=3)
    elif pick_me.endswith('.png'):
      image = tf.io.decode_png(image, channels=3)
    elif pick_me.endswith('.bmp'):
      image = tf.io.decode_bmp(image, channels=3)

    image = tf.image.resize(image, [width, height])
    image = tf.cast(image / 255., tf.float32)
    image = tf.expand_dims(image, 0)
    yield [image]
```

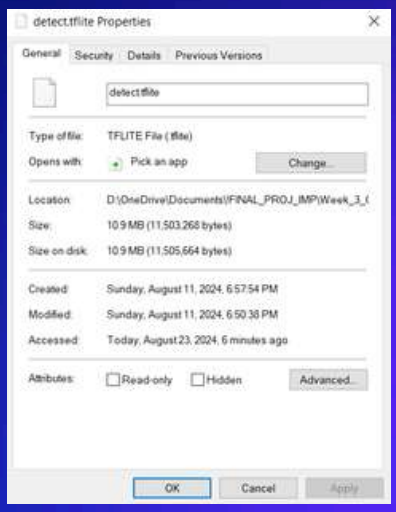
```python
# Initialize converter module
converter = tf.lite.TFLiteConverter.from_saved_model('/content/custom_model_lite/saved_model')

# enables quantization
converter.optimizations = [tf.lite.Optimize.DEFAULT]
# sets representative dataset for quantization
converter.representative_dataset = representative_data_gen
# Prevent the converter throws an error
converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS, tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
# For full int8 quantization
converter.target_spec.supported_types = [tf.int8]
# These set the input tensors to uint8 and output tensors to float32
converter.inference_input_type = tf.uint8
converter.inference_output_type = tf.float32
tflite_model = converter.convert()

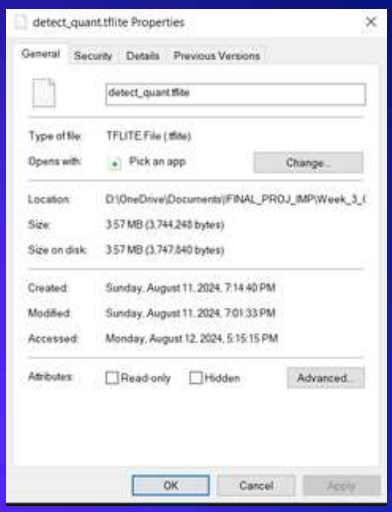
with open('/content/custom_model_lite/detect_quant.tflite', 'wb') as f:
  f.write(tflite_model)
```

Quantization Model

Comparing



BEFORE

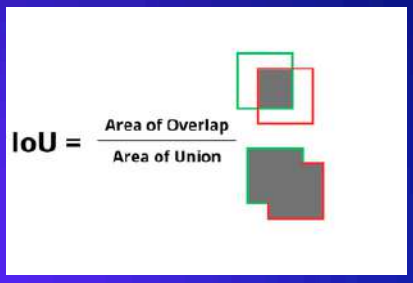


AFTER

Evaluate Model

สำหรับ Object Detection แทนที่จะใช้ ***"Confusion Matrix"*** ซึ่งใช้ทั่วไปใน Image Classification เราจะใช้ตัวชี้วัดอย่าง Precision หรือ Recall ในรูปแบบ**เฉลี่ย**ที่เรียกว่า **Mean Average Precision (mAP)** ซึ่งจะผสมผสานกับตัวชี้วัด Intersection over Union (IoU) โดย IoU จะถูกคำนวณโดยการนำพื้นที่ที่สองกรอบบรรจุวัตถุ (bounding boxes) ซ้อนทับกันมาเป็นตัวเศษ และนำพื้นที่รวมของสองกรอบบรรจุวัตถุมาเป็นตัวส่วน ซึ่งในรูปแบบทางคณิตศาสตร์ สูตรของ IoU สามารถเขียนได้ดังนี้:

$$IoU = \frac{(\text{Area of Intersection})}{(\text{Area of Union})} = \frac{|A \cap}{|A \cup}$$



สูตรนี้ช่วยหาค่าจำนวนที่บอกถึงระดับการซ้อนทับระหว่าง กรอบบรรจุวัตถุที่ทำนาย (predicted bounding box) กับ กรอบบรรจุวัตถุจริง (ground truth bounding box) โดยให้ค่าตัวเลขที่บ่งบอกว่าโมเดลทำนายตำแหน่งของวัตถุได้ตรงกับตำแหน่งจริงมากเพียงใด โดยค่าที่สูงของ IoU แสดงถึงความสอดคล้องระหว่างกรอบที่ทำนายและกรอบจริง ส่วนค่าที่ต่ำของ IoU หมายถึงความคลาดเคลื่อนระหว่างกรอบที่ทำนายกับกรอบจริง

Evaluate Model

Result

```
[WinError 2] The system cannot find the file specified: './mAP'
d:\OneDrive\Documents\!FINAL_PROJ_IMP\Week_3_Cus_DATA\custom_model_lite\mAP
Calculating mAP at 0.50 IoU threshold...
94.66% = mask_weared_incorrect AP
71.23% = with_mask AP
36.34% = without_mask AP
mAP = 67.41%

***mAP Results***

Class           Average mAP @ 0.5:0.95
---------------------------------------
with_mask               71.23%
without_mask            36.34%
mask_weared_incorrect           94.66%

Overall         67.41%
```

detect.tflite
(No Quantization)

```
D:\OneDrive\Documents\!FINAL_PROJ_IMP\Week_3_Cus_DATA\custom_model_lite\mAP
Calculating mAP at 0.50 IoU threshold...
96.03% = mask_weared_incorrect AP
67.97% = with_mask AP
31.89% = without_mask AP
mAP = 65.30%

***mAP Results***

Class           Average mAP @ 0.5:0.95
---------------------------------------
with_mask               67.97%
without_mask            31.89%
mask_weared_incorrect           96.03%

Overall         65.30%
```

detect_quant.tflite
(Quantization)

ใช้ Coral กับ กล้อง

หลังจากเชื่อมต่อ Coral กับ Coral Camera และตรวจสอบด้วยคำสั่ง v4l2 ดูเหมือนว่าทุกอย่างจะทำงานได้ตามปกติจากผลลัพธ์ที่แสดงว่า Coral Board สามารถตรวจจับ Coral Camera ได้ และสามารถใช้งานได้ นอกจากนี้ยังสามารถถ่ายภาพด้วยคำสั่ง snapshot ได้ด้วย

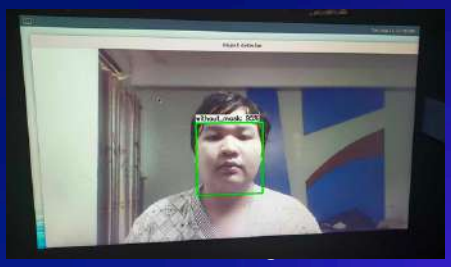


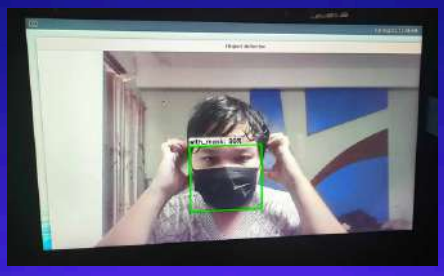
```
$ v4l2-ctl --list-formats-ext --device /dev/video1
ioctl: VIDIOC_ENUM_FMT
        Type: Video Capture

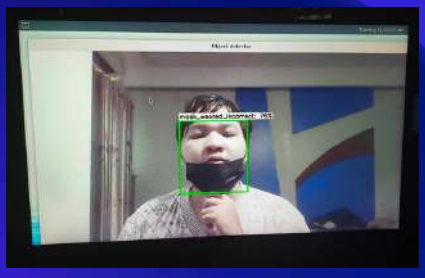
        [0]: 'YUYV' (YUYV 4:2:2)
                Size: Discrete 640x480
                        Interval: Discrete 0.033s (30.000 fps)
                Size: Discrete 720x480
                        Interval: Discrete 0.033s (30.000 fps)
                Size: Discrete 1280x720
                        Interval: Discrete 0.033s (30.000 fps)
                Size: Discrete 1920x1080
                        Interval: Discrete 0.067s (15.000 fps)
                        Interval: Discrete 0.033s (30.000 fps)
                Size: Discrete 2592x1944
                        Interval: Discrete 0.067s (15.000 fps)
```

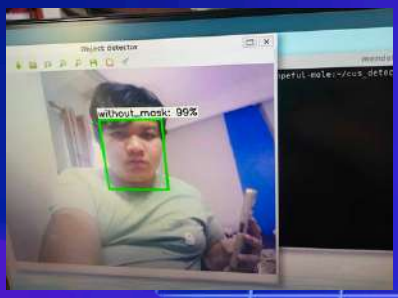
ใช้ Cobot กับกล้อง

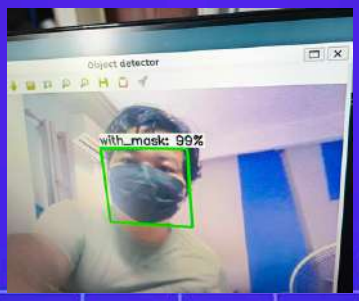
Video Testing Result













ปรับ FPS Coral Camera

หลังจากทุกอย่างเสร็จสิ้นและรันไฟล์แล้ว พบว่ามีปัญหาด้าน FPS ที่ต่ำกว่า 10 FPS แต่ในเอกสารข้อมูลของ Coral Camera ระบุว่ากล้องสามารถรองรับความละเอียดสูงสุดที่ 1944p30 แต่โค้ดที่ใช้ร่วมกับ TPU นั้นทำได้เพียง 10 FPS และมีความล่าช้า อาจเป็นเพราะรูปแบบ YUYV ที่ไม่รองรับกับ OpenCV โดยตรงเหมือนกับ MJPEG โดยสามารถแก้ไขได้โดยการลดการใช้ if-else statement ในตัว Code ลง เนื่องจาก if-else statement นั้นต้องเช็คตลอดเวลา ทำให้ต้องแบ่งทรัพยากรเครื่องส่วนหนึ่งไปไว้ตรวจ จึงนำออก และกำหนดไปโดยตรงแทน รวมถึงต้องกำหนด Width, Height และ FPS ให้ตรงกับที่ v4l2 ได้พบ และเปลี่ยน Format ของภาพที่ได้ เป็น MJPG เพื่อให้ตรงกับ Format ของ OpenCV

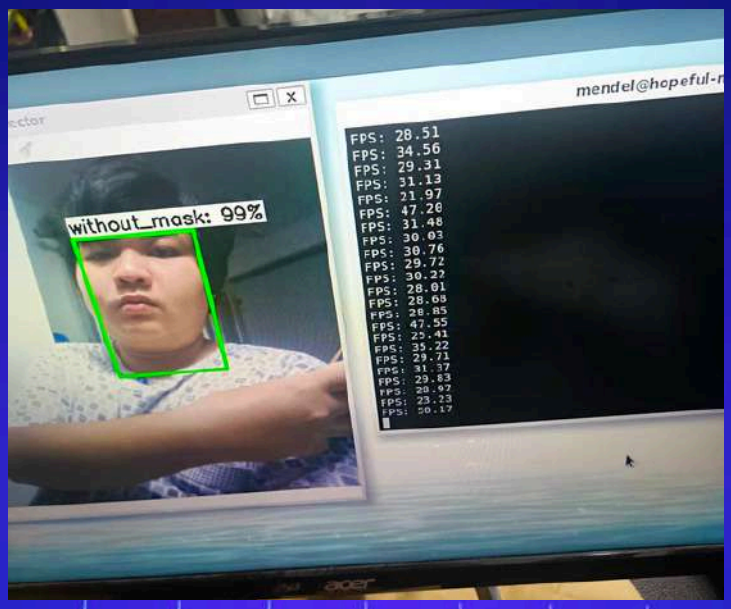
```
$ v4l2-ctl --list-formats-ext --device /dev/video1
ioctl: VIDIOC_ENUM_FMT
        Type: Video Capture

        [0]: 'YUYV' (YUYV 4:2:2)
                Size: Discrete 640x480
                        Interval: Discrete 0.033s (30.000 fps)
                Size: Discrete 720x480
                        Interval: Discrete 0.033s (30.000 fps)
                Size: Discrete 1280x720
                        Interval: Discrete 0.033s (30.000 fps)
                Size: Discrete 1920x1080
                        Interval: Discrete 0.067s (15.000 fps)
                        Interval: Discrete 0.033s (30.000 fps)
                Size: Discrete 2592x1944
                        Interval: Discrete 0.067s (15.000 fps)
```

```python
# Open video capture stream from camera
video = cv2.VideoCapture(1, cv2.CAP_V4L2)

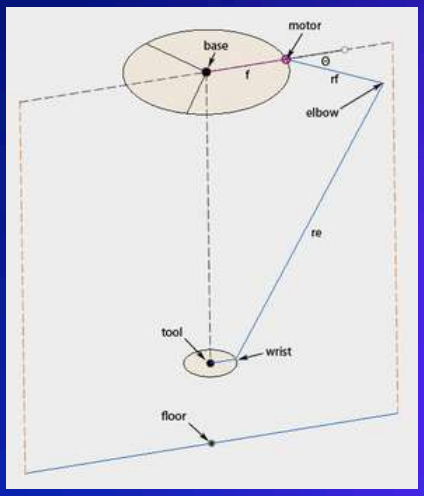
# Set camera resolution to 640x480 and 30fps
video.set(cv2.CAP_PROP_FOURCC, cv2.VideoWriter_fourcc(*'MJPG'))
video.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
video.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
video.set(cv2.CAP_PROP_FPS, 30)
```

Using FPS Coral Camera



Math Real Rotation(Theta). from x, y bbox

เป็นการทดลองนำสูตรคำนวณ θ1, θ2, θ3 จาก Bounding Box ที่ Detect ได้ โดยการหา Center Point และนำไปคำนวณมุม ด้วย Inverse Kinematics Function โดยจะนำ x, y, z เข้าไป และส่ง θ1, θ2, θ3 ที่คำนวณได้ออกมา ในที่นี้ได้นำตัว Code มากจาก **Delta robot - Inverse and forward kinematics library - (For @bitbeam bot)** — **https://gist.github.com/hugs/4231272**

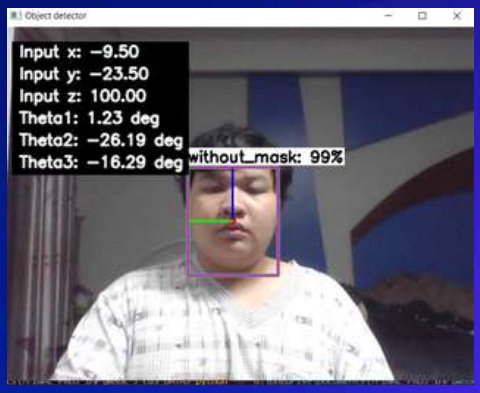


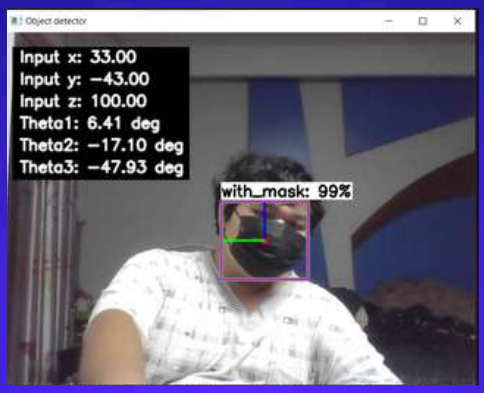

```
# Specific geometry for Delta Arm:
e  =  26.0 # End Effector radius(mm)
f  =  69.0 # Base radius(mm)
re = 128.0 # Forearm length(mm)
rf =  88.0 # Bicep length(mm)
```

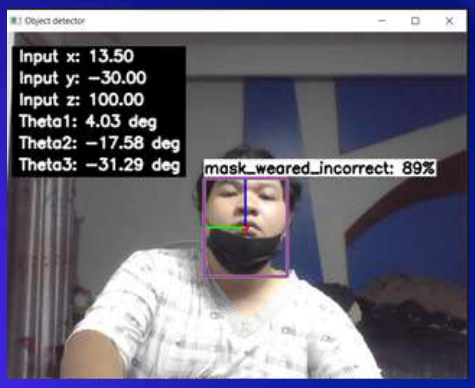
Math Real Rotation(Theta). from x, y bbox

Testing Result

ในที่นี้กำหนด z = 100 เพื่อทดสอบ







ทดสอบ Motor ด้วยโซ่ และวัด Force 45 deg วัด Payload






THANK YOU
FOR YOUR ATTENTION!